```
# ============================================================
# CUSTOMER CHURN PREDICTION AND CUSTOMER SEGMENTATION
# IBM TELCO CUSTOMER CHURN DATASET
# GOOGLE COLAB COMPLETE CODE
# ============================================================

# ============================================================
# 1. IMPORT LIBRARIES
# ============================================================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer

from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)

from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.base import BaseEstimator, TransformerMixin


# ============================================================
# 2. LOAD DATASET
# ============================================================

URL = "https://raw.githubusercontent.com/IBM/telco-customer-churn-on-icp4d/master/data/Telco-Custo

df = pd.read_csv(URL)

print("Dataset loaded successfully!")
print("Dataset shape:", df.shape)

display(df.head())


# ============================================================
# 3. DATA INFORMATION
# ============================================================

print("\nDataset Information:")
df.info()

print("\nMissing Values:")
print(df.isnull().sum())


# ============================================================
# 4. DATA CLEANING
```

```
# ============================================================

# Convert TotalCharges to numeric
df["TotalCharges"] = pd.to_numeric(
    df["TotalCharges"],
    errors="coerce"
)

# Convert Churn:
# No = 0
# Yes = 1
df["Churn"] = df["Churn"].map({
    "No": 0,
    "Yes": 1
})

print("\nCleaned Dataset:")
display(df.head())


# ============================================================
# 5. CHURN DISTRIBUTION
# ============================================================

churn_counts = df["Churn"].value_counts()

plt.figure(figsize=(7, 5))

plt.bar(
    ["Non-Churn", "Churn"],
    [
        churn_counts.get(0, 0),
        churn_counts.get(1, 0)
    ]
)

plt.xlabel("Customer Status")
plt.ylabel("Number of Customers")
plt.title("Customer Churn Distribution")

plt.show()


# ============================================================
# 6. CONTRACT DISTRIBUTION
# ============================================================

contract_share = (
    df["Contract"]
    .value_counts(normalize=True)
    * 100
)

plt.figure(figsize=(7, 5))

plt.bar(
    contract_share.index,
    contract_share.values
)

plt.xlabel("Contract Type")
plt.ylabel("Share (%)")
plt.title("Customers by Contract Type")

plt.xticks(rotation=15)

plt.show()
```

```
# ============================================================
# 7. CHURN RATE BY CONTRACT
# ============================================================

contract_churn = (
    df.groupby("Contract")["Churn"]
    .mean()
    * 100
)

plt.figure(figsize=(7, 5))

plt.bar(
    contract_churn.index,
    contract_churn.values
)

plt.xlabel("Contract Type")
plt.ylabel("Churn Rate (%)")
plt.title("Churn Rate by Contract Type")

plt.xticks(rotation=15)

plt.show()


# ============================================================
# 8. CHURN RATE BY INTERNET SERVICE
# ============================================================

internet_churn = (
    df.groupby("InternetService")["Churn"]
    .mean()
    * 100
)

plt.figure(figsize=(7, 5))

plt.bar(
    internet_churn.index,
    internet_churn.values
)

plt.xlabel("Internet Service")
plt.ylabel("Churn Rate (%)")
plt.title("Churn Rate by Internet Service")

plt.xticks(rotation=15)

plt.show()


# ============================================================
# 9. PREPARE DATA FOR CLASSIFICATION
# ============================================================

# Remove Churn and customerID from input features
X = df.drop(
    columns=["Churn", "customerID"]
)

# Target
y = df["Churn"]


# ============================================================
```

```
# 10. DEFINE NUMERICAL AND CATEGORICAL FEATURES
# ============================================================

numeric_features = [
    "SeniorCitizen",
    "tenure",
    "MonthlyCharges",
    "TotalCharges"
]

categorical_features = [
    column
    for column in X.columns
    if column not in numeric_features
]


# ============================================================
# 11. NUMERICAL PIPELINE
# ============================================================

numeric_pipe = Pipeline([
    (
        "imputer",
        SimpleImputer(strategy="median")
    ),
    (
        "scaler",
        StandardScaler()
    )
])


# ============================================================
# 12. CATEGORICAL PIPELINE
# ============================================================

categorical_pipe = Pipeline([
    (
        "imputer",
        SimpleImputer(strategy="most_frequent")
    ),
    (
        "onehot",
        OneHotEncoder(handle_unknown="ignore")
    )
])


# ============================================================
# 13. COMBINE PREPROCESSING
# ============================================================

preprocessor = ColumnTransformer([
    (
        "num",
        numeric_pipe,
        numeric_features
    ),
    (
        "cat",
        categorical_pipe,
        categorical_features
    )
])


# ------------------------------------------------------------
```

```
# ============================================================
# 14. TRAIN TEST SPLIT
# ============================================================

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\nTraining samples:", len(X_train))
print("Testing samples:", len(X_test))


# ============================================================
# 15. DENSE TRANSFORMER
# ============================================================

class DenseTransformer(BaseEstimator, TransformerMixin):

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        if hasattr(X, "toarray"):
            return X.toarray()
        return X


# ============================================================
# 16. MACHINE LEARNING MODELS
# ============================================================

models = {
    "Logistic Regression": LogisticRegression(
        max_iter=2000
    ),

    "Gaussian Naive Bayes": GaussianNB(),

    "Decision Tree": DecisionTreeClassifier(
        max_depth=5,
        min_samples_leaf=10,
        random_state=42
    )
}


# ============================================================
# 17. TRAIN MODELS
# ============================================================

results = []

trained_models = {}

predictions = {}


for name, model in models.items():

    steps = [
        ("prep", preprocessor)
    ]

    if name == "Gaussian Naive Bayes":
```

```
        steps.append(
            ("dense", DenseTransformer())
        )

    steps.append(
        ("model", model)
    )

    clf = Pipeline(steps)

    # Train
    clf.fit(
        X_train,
        y_train
    )

    # Predict
    pred = clf.predict(
        X_test
    )

    trained_models[name] = clf
    predictions[name] = pred

    # Metrics
    accuracy = accuracy_score(
        y_test,
        pred
    )

    precision = precision_score(
        y_test,
        pred,
        zero_division=0
    )

    recall = recall_score(
        y_test,
        pred,
        zero_division=0
    )

    f1 = f1_score(
        y_test,
        pred,
        zero_division=0
    )

    results.append([
        name,
        accuracy,
        precision,
        recall,
        f1
    ])

    print("\n========================================")
    print(name)
    print("========================================")

    print(
        classification_report(
            y_test,
            pred,
            target_names=[
                "Non-Churn",
                "Churn"
```

```
            ],
            zero_division=0
        )
    )

    print("Confusion Matrix:")

    print(
        confusion_matrix(
            y_test,
            pred
        )
    )


# ============================================================
# 18. MODEL RESULTS TABLE
# ============================================================

results_df = pd.DataFrame(
    results,
    columns=[
        "Model",
        "Accuracy",
        "Precision",
        "Recall",
        "F1-score"
    ]
)

print("\nMODEL PERFORMANCE")

display(results_df)


# ============================================================
# 19. MODEL ACCURACY GRAPH
# ============================================================

plt.figure(figsize=(8, 5))

plt.bar(
    results_df["Model"],
    results_df["Accuracy"] * 100
)

plt.xlabel("Model")
plt.ylabel("Accuracy (%)")
plt.title("Classification Accuracy Comparison")

plt.xticks(rotation=15)

plt.show()


# ============================================================
# 20. LOGISTIC REGRESSION CONFUSION MATRIX
# ============================================================

lr_pred = predictions["Logistic Regression"]

cm = confusion_matrix(
    y_test,
    lr_pred
)

plt.figure(figsize=(7, 6))
```

```
plt.imshow(cm)

plt.title(
    "Logistic Regression Confusion Matrix"
)

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.xticks(
    [0, 1],
    ["Non-Churn", "Churn"]
)

plt.yticks(
    [0, 1],
    ["Non-Churn", "Churn"]
)

for i in range(2):
    for j in range(2):

        plt.text(
            j,
            i,
            cm[i, j],
            ha="center",
            va="center"
        )

plt.colorbar()

plt.show()


# ============================================================
# 21. K-MEANS CLUSTERING
# ============================================================

cluster_num = [
    "SeniorCitizen",
    "tenure",
    "MonthlyCharges",
    "TotalCharges"
]

cluster_cat = [
    "PhoneService",
    "MultipleLines",
    "InternetService",
    "OnlineSecurity",
    "OnlineBackup",
    "DeviceProtection",
    "TechSupport",
    "StreamingTV",
    "StreamingMovies",
    "Contract",
    "PaperlessBilling",
    "PaymentMethod"
]


# ============================================================
# 22. CLUSTER PREPROCESSING
# ============================================================
```

```
cluster_preprocessor = ColumnTransformer([

    (
        "num",

        Pipeline([
            (
                "imputer",
                SimpleImputer(
                    strategy="median"
                )
            ),

            (
                "scale",
                StandardScaler()
            )
        ]),

        cluster_num
    ),

    (
        "cat",

        Pipeline([
            (
                "imputer",
                SimpleImputer(
                    strategy="most_frequent"
                )
            ),

            (
                "onehot",
                OneHotEncoder(
                    handle_unknown="ignore"
                )
            )
        ]),

        cluster_cat
    )
])


# ============================================================
# 23. CREATE CLUSTER DATA
# ============================================================

X_cluster = (
    cluster_preprocessor
    .fit_transform(df)
)

print(
    "\nCluster data shape:",
    X_cluster.shape
)


# ============================================================
# 24. ELBOW METHOD
# ============================================================

inertias = []

for k in range(2, 9):
```

```
    km = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=20
    )

    km.fit(X_cluster)

    inertias.append(
        km.inertia_
    )


plt.figure(figsize=(8, 5))

plt.plot(
    range(2, 9),
    inertias,
    marker="o"
)

plt.xlabel("K")
plt.ylabel("Inertia")

plt.title("Elbow Method")

plt.show()


# ============================================================
# 25. K-MEANS WITH 4 CLUSTERS
# ============================================================

kmeans = KMeans(
    n_clusters=4,
    random_state=42,
    n_init=20
)

df["Cluster"] = kmeans.fit_predict(
    X_cluster
)


# ============================================================
# 26. CUSTOMER CLUSTER PROFILE
# ============================================================

profile = (
    df.groupby("Cluster")
    .agg(
        Customers=(
            "customerID",
            "count"
        ),

        AvgTenure=(
            "tenure",
            "mean"
        ),

        AvgMonthlyCharges=(
            "MonthlyCharges",
            "mean"
        ),
```

```
        ChurnRate=(
            "Churn",
            "mean"
        )
    )
    .round(3)
)

print("\nCUSTOMER CLUSTER PROFILE")

display(profile)


# ============================================================
# 27. PCA
# ============================================================

X_dense = (
    X_cluster.toarray()
    if hasattr(X_cluster, "toarray")
    else X_cluster
)

pca = PCA(
    n_components=2,
    random_state=42
)

X_pca = pca.fit_transform(
    X_dense
)


# ============================================================
# 28. PCA CLUSTER VISUALIZATION
# ============================================================

plt.figure(figsize=(8, 6))

for c in sorted(
    df["Cluster"].unique()
):

    mask = (
        df["Cluster"] == c
    )

    plt.scatter(
        X_pca[mask, 0],
        X_pca[mask, 1],
        s=12,
        label=f"Cluster {c}"
    )


plt.xlabel(
    "Principal Component 1"
)

plt.ylabel(
    "Principal Component 2"
)

plt.title(
    "Customer Clusters in PCA Space"
)

plt.legend()
```

```
plt.legend()

plt.show()


# ============================================================
# 29. PCA EXPLAINED VARIANCE
# ============================================================

print("\nPCA Explained Variance:")

print(
    pca.explained_variance_ratio_
)


# ============================================================
# 30. FINAL OUTPUT
# ============================================================

print("\n")
print("========================================")
print("PROJECT COMPLETED SUCCESSFULLY")
print("========================================")

print("\nClassification Results:")
display(results_df)

print("\nCluster Profile:")
display(profile)

print(
    "\nPreferred baseline model: Logistic Regression"
)

print(
    "Number of clusters: 4"
)
```

```
Dataset loaded successfully!
Dataset shape: (7043, 21)
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	In
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	
1	5575-GNVDE	Male	0	No	No	34	Yes	No	
2	3668-QPYBK	Male	0	No	No	2	Yes	No	
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	
4	9237-HQITU	Female	0	No	No	2	Yes	No	

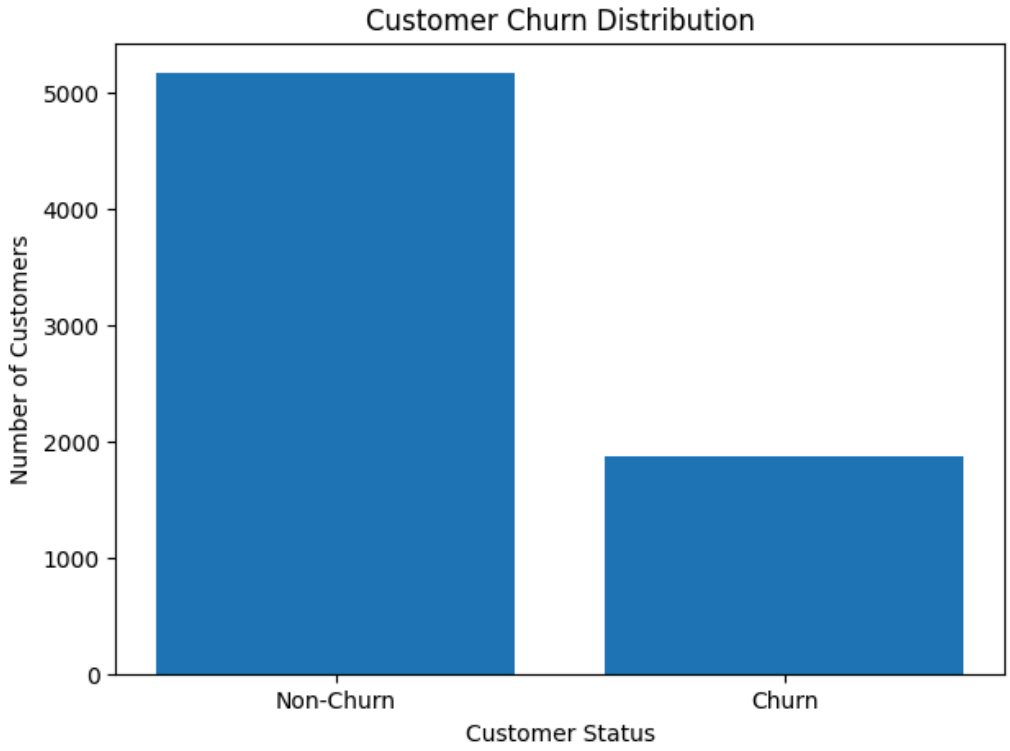
5 rows × 21 columns

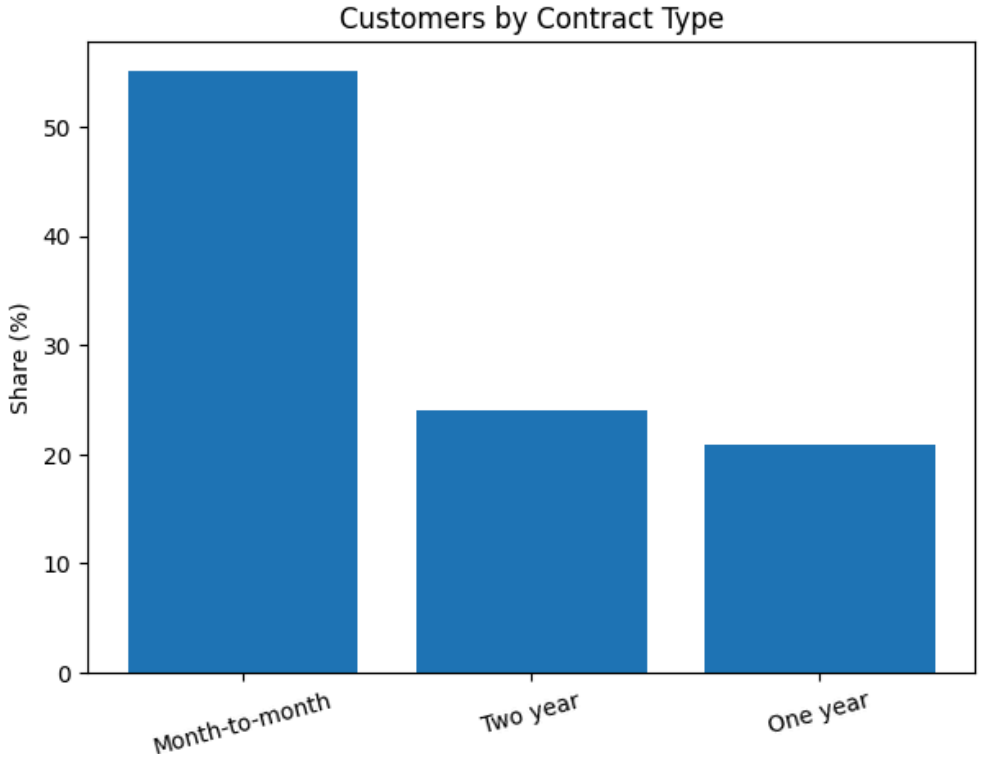
```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customerID        7043 non-null   object
 1   gender            7043 non-null   object
 2   SeniorCitizen     7043 non-null   int64
 3   Partner           7043 non-null   object
 4   Dependents        7043 non-null   object
 5   tenure            7043 non-null   int64
 6   PhoneService      7043 non-null   object
 7   MultipleLines     7043 non-null   object
 8   InternetService   7043 non-null   object
 9   OnlineSecurity    7043 non-null   object
 10  OnlineBackup      7043 non-null   object
 11  DeviceProtection  7043 non-null   object
 12  TechSupport       7043 non-null   object
 13  StreamingTV       7043 non-null   object
 14  StreamingMovies   7043 non-null   object
 15  Contract          7043 non-null   object
 16  PaperlessBilling  7043 non-null   object
 17  PaymentMethod     7043 non-null   object
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   object
 20  Churn             7043 non-null   object
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB

Missing Values:
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges        0
Churn               0
dtype: int64
```

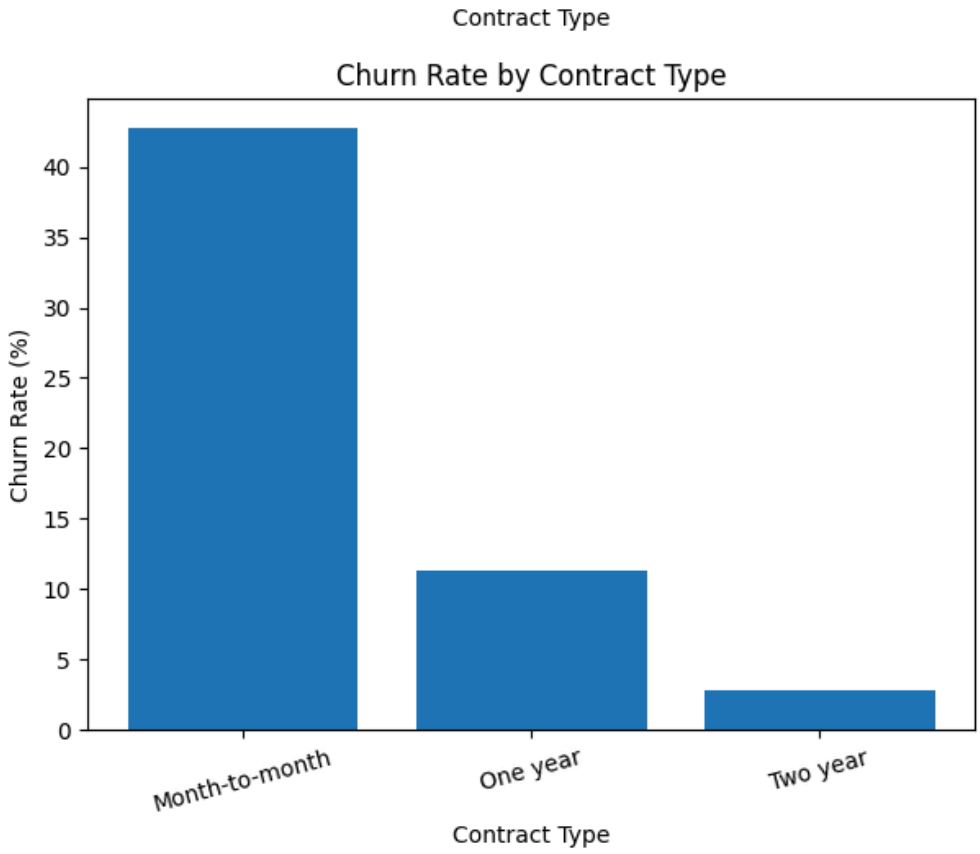
Cleaned Dataset:

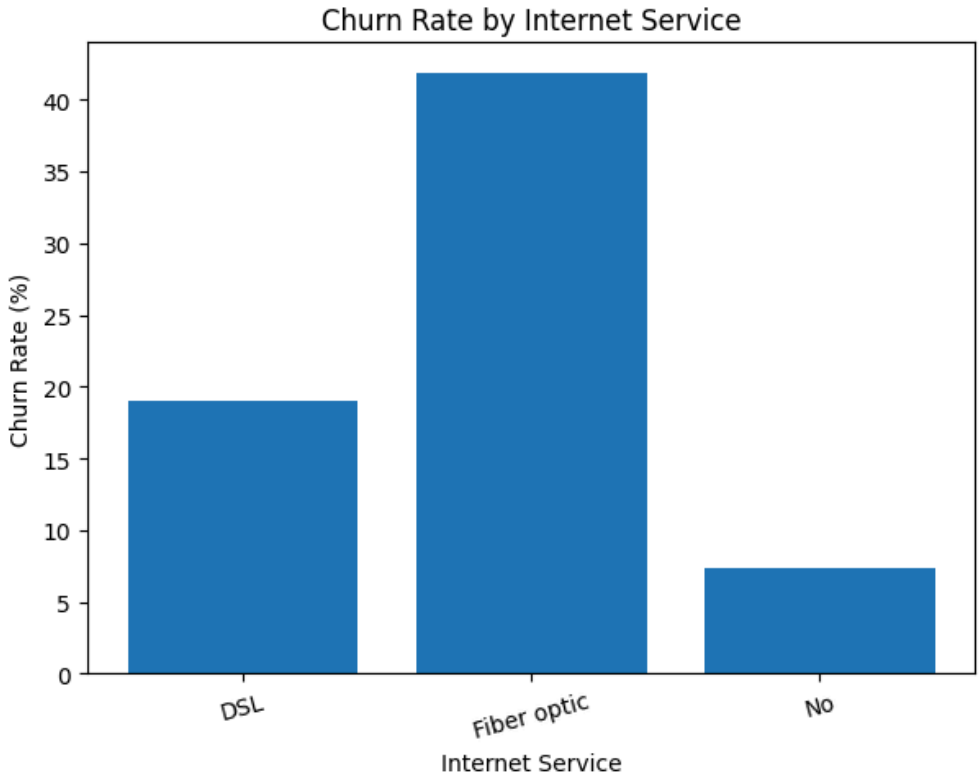
	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	In
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	
1	5575-GNVDE	Male	0	No	No	34	Yes	No	
2	3668-QPYBK	Male	0	No	No	2	Yes	No	
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	
4	9237-HQITU	Female	0	No	No	2	Yes	No	

5 rows × 21 columns





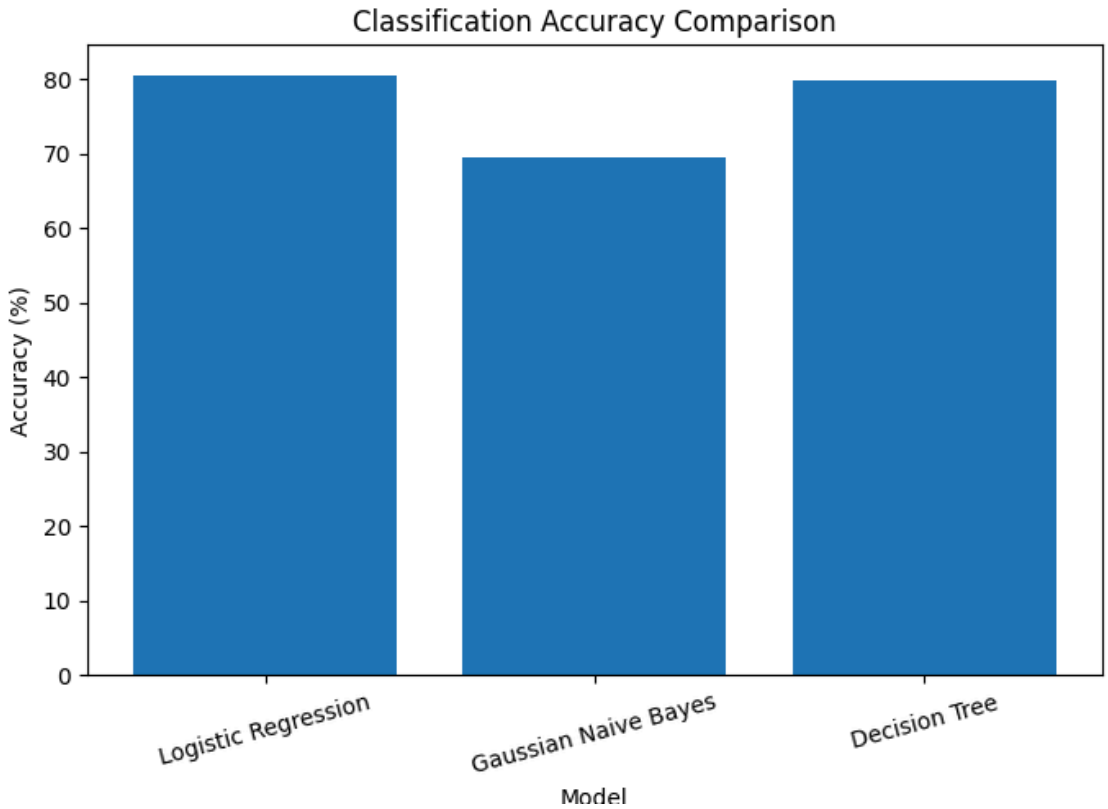


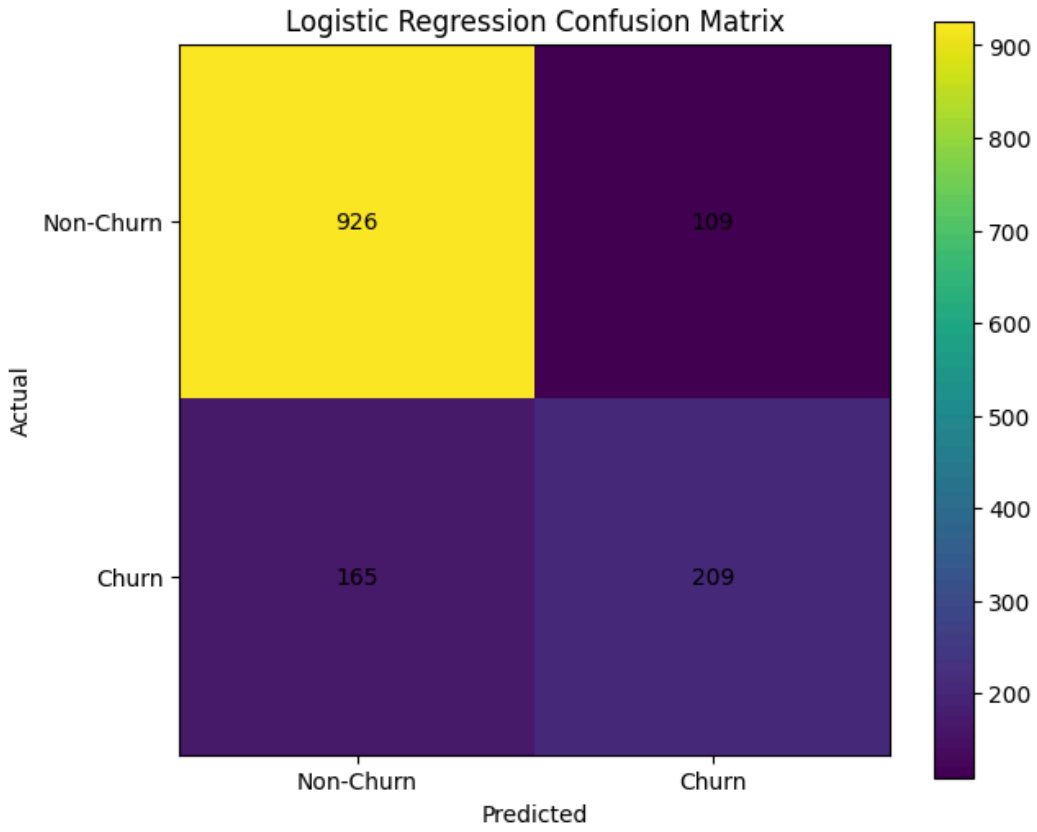



```
Training samples: 5634
Testing samples: 1409

========================================
Logistic Regression
========================================
              precision    recall  f1-score   support

   Non-Churn       0.85      0.89      0.87      1035
       Churn       0.66      0.56      0.60       374

    accuracy                           0.81      1409
   macro avg       0.75      0.73      0.74      1409
weighted avg       0.80      0.81      0.80      1409
```

```
Confusion Matrix:
[[926 109]
 [165 209]]

========================================
Gaussian Naive Bayes
========================================
              precision    recall  f1-score   support

   Non-Churn       0.92      0.64      0.76      1035
       Churn       0.46      0.84      0.59       374

    accuracy                           0.69      1409
   macro avg       0.69      0.74      0.67      1409
weighted avg       0.79      0.69      0.71      1409

Confusion Matrix:
[[666 369]
 [ 61 313]]

========================================
Decision Tree
========================================
              precision    recall  f1-score   support

   Non-Churn       0.85      0.88      0.87      1035
       Churn       0.63      0.57      0.60       374

    accuracy                           0.80      1409
   macro avg       0.74      0.72      0.73      1409
weighted avg       0.79      0.80      0.79      1409

Confusion Matrix:
[[913 122]
 [162 212]]

MODEL PERFORMANCE
```

	Model	Accuracy	Precision	Recall	F1-score
0	Logistic Regression	0.805536	0.657233	0.558824	0.604046
1	Gaussian Naive Bayes	0.694819	0.458944	0.836898	0.592803
2	Decision Tree	0.798439	0.634731	0.566845	0.598870






```
Cluster data shape: (7043, 39)
```